JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class AtomicLongArray

java.lang.Object
    java.util.concurrent.atomic.AtomicLongArray

All Implemented Interfaces:

Serializable

```
public class AtomicLongArray
extends Object
implements Serializable
```

A long array in which elements may be updated atomically. See the VarHandle specification for descriptions of the properties of atomic accesses.

Since:

1.5

See Also:

Serialized Form

Constructor Summary

Constructors

Constructor	Description
`AtomicLongArray(int length)`	Creates a new AtomicLongArray of the given length, with all elements initially zero.
`AtomicLongArray(long[] array)`	Creates a new AtomicLongArray with the same length as, and all elements copied from, the given array.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
final long	`accumulateAndGet(int i, long x, LongBinaryOperator accum`	Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the element at index `i` with the results of

		applying the given function to the current and given values, returning the updated value.
long	**addAndGet**(int i, long delta)	Atomically adds the given value to the element at index i, with memory effects as specified by VarHandle.getAndAdd(Object...).
final long	**compareAndExchange**(int i, long expectedValue, long newValue)	Atomically sets the element at index i to newValue if the element's current value, referred to as the *witness value*, == expectedValue, with memory effects as specified by VarHandle.compareAndExchange(Object...).
final long	**compareAndExchangeAcquir**(int i, long expectedValue, long newValue)	Atomically sets the element at index i to newValue if the element's current value, referred to as the *witness value*, == expectedValue, with memory effects as specified by VarHandle.compareAndExchangeAcquire(Obje
final long	**compareAndExchangeReleas**(int i, long expectedValue, long newValue)	Atomically sets the element at index i to newValue if the element's current value, referred to as the *witness value*, == expectedValue, with

		memory effects as specified by VarHandle.compareAndExchangeRelease(Obje
final boolean	**compareAndSet**(int i, long expectedValue, long newValue)	Atomically sets the element at index i to newValue if the element's current value == expectedValue, with memory effects as specified by VarHandle.compareAndSet(Object...).
final long	**decrementAndGet**(int i)	Atomically decrements the value of the element at index i, with memory effects as specified by VarHandle.getAndAdd(Object...).
final long	**get**(int i)	Returns the current value of the element at index i, with memory effects as specified by VarHandle.getVolatile(Object...).
final long	**getAcquire**(int i)	Returns the current value of the element at index i, with memory effects as specified by VarHandle.getAcquire(Object...).
final long	**getAndAccumulate**(int i, long x, LongBinaryOperator accum	Atomically updates (with memory effects as specified by VarHandle.compareAndSet(Object...)) the element at index i with the results of applying the given function to the current

		and given values, returning the previous value.
final long	**getAndAdd**(int i, long delta)	Atomically adds the given value to the element at index `i`, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.
final long	**getAndDecrement**(int i)	Atomically decrements the value of the element at index `i`, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.
final long	**getAndIncrement**(int i)	Atomically increments the value of the element at index `i`, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.
final long	**getAndSet**(int i, long newValue)	Atomically sets the element at index `i` to `newValue` and returns the old value, with memory effects as specified by `VarHandle.getAndSet(Object...)`.
final long	**getAndUpdate**(int i, LongUnaryOperator update	Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the element at index `i` with the results of

		applying the given function, returning the previous value.
final long	**getOpaque**(int i)	Returns the current value of the element at index i, with memory effects as specified by VarHandle.getOpaque(Object...).
final long	**getPlain**(int i)	Returns the current value of the element at index i, with memory semantics of reading as if the variable was declared non-volatile.
final long	**incrementAndGet**(int i)	Atomically increments the value of the element at index i, with memory effects as specified by VarHandle.getAndAdd(Object...).
final void	**lazySet**(int i, long newValue)	Sets the element at index i to newValue, with memory effects as specified by VarHandle.setRelease(Object...).
final int	**length**()	Returns the length of the array.
final void	**set**(int i, long newValue)	Sets the element at index i to newValue, with memory effects as specified by VarHandle.setVolatile(Object...).
final void	**setOpaque**(int i, long newValue)	Sets the element at index i to newValue, with

		memory effects as specified by `VarHandle.setOpaque(Object...)`.
`final void`	**`setPlain`**`(int i, long newValue)`	Sets the element at index `i` to `newValue`, with memory semantics of setting as if the variable was declared non-`volatile` and non-`final`.
`final void`	**`setRelease`**`(int i, long newValue)`	Sets the element at index `i` to `newValue`, with memory effects as specified by `VarHandle.setRelease(Object...)`.
`String`	**`toString`**`()`	Returns the String representation of the current values of array.
`final long`	**`updateAndGet`**`(int i, LongUnaryOperator update`	Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the element at index `i` with the results of applying the given function, returning the updated value.
`final boolean`	**`weakCompareAndSet`**`(int i, long expectedValue, long newValue)`	**Deprecated.** This method has plain memory effects but the method name implies volatile memory effects (see methods such as `compareAndExchange(int, long, long)` and `compareAndSet(int, long, long)`).
`final boolean`	**`weakCompareAndSetAcquire`**`(int i,`	Possibly atomically sets the element at index `i` to

	long expectedValue, long newValue)	newValue if the element's current value == expectedValue, with memory effects as specified by VarHandle.weakCompareAndSetAcquire(Objec
final boolean	**weakCompareAndSetPlain** (int i, long expectedValue, long newValue)	Possibly atomically sets the element at index i to newValue if the element's current value == expectedValue, with memory effects as specified by VarHandle.weakCompareAndSetPlain(Object.
final boolean	**weakCompareAndSetRelease** (int i, long expectedValue, long newValue)	Possibly atomically sets the element at index i to newValue if the element's current value == expectedValue, with memory effects as specified by VarHandle.weakCompareAndSetRelease(Objec
final boolean	**weakCompareAndSetVolatil** (int i, long expectedValue, long newValue)	Possibly atomically sets the element at index i to newValue if the element's current value == expectedValue, with memory effects as specified by VarHandle.weakCompareAndSet(Object...).

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Constructor Details

AtomicLongArray

```
public AtomicLongArray(int length)
```

Creates a new AtomicLongArray of the given length, with all elements initially zero.

Parameters:

`length` - the length of the array

AtomicLongArray

```
public AtomicLongArray(long[] array)
```

Creates a new AtomicLongArray with the same length as, and all elements copied from, the given array.

Parameters:

`array` - the array to copy elements from

Throws:

`NullPointerException` - if array is null

Method Details

length

```
public final int length()
```

Returns the length of the array.

Returns:

the length of the array

get

```
public final long get(int i)
```

Returns the current value of the element at index `i`, with memory effects as specified by `VarHandle.getVolatile(Object...)`.

Parameters:

i - the index

Returns:

the current value

set

```
public final void set(int i,
                      long newValue)
```

Sets the element at index `i` to `newValue`, with memory effects as specified by `VarHandle.setVolatile(Object...)`.

Parameters:

`i` - the index

`newValue` - the new value

lazySet

```
public final void lazySet(int i,
                          long newValue)
```

Sets the element at index `i` to `newValue`, with memory effects as specified by `VarHandle.setRelease(Object...)`.

Parameters:

`i` - the index

`newValue` - the new value

Since:

1.6

getAndSet

```
public final long getAndSet(int i,
                            long newValue)
```

Atomically sets the element at index `i` to `newValue` and returns the old value, with memory effects as specified by `VarHandle.getAndSet(Object...)`.

Parameters:

`i` - the index

`newValue` - the new value

Returns:

the previous value

compareAndSet

```
public final boolean compareAndSet(int i,
                                   long expectedValue,
                                   long newValue)
```

Atomically sets the element at index `i` to `newValue` if the element's current value == `expectedValue`, with memory effects as specified by `VarHandle.compareAndSet(Object...)`.

Parameters:

`i` - the index

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful. False return indicates that the actual value was not equal to the expected value.

weakCompareAndSet

```
@Deprecated(since="9")
public final boolean weakCompareAndSet(int i,
                                       long expectedValue,
                                       long newValue)
```

Deprecated.
This method has plain memory effects but the method name implies volatile memory effects (see methods such as `compareAndExchange(int, long, long)` and `compareAndSet(int, long, long)`). To avoid confusion over plain or volatile memory effects it is recommended that the method `weakCompareAndSetPlain(int, long, long)` be used instead.

Possibly atomically sets the element at index `i` to `newValue` if the element's current value == `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSetPlain(Object...)`.

Parameters:

`i` - the index

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

See Also:

`weakCompareAndSetPlain(int, long, long)`

weakCompareAndSetPlain

```
public final boolean weakCompareAndSetPlain(int i,
                                            long expectedValue,
                                            long newValue)
```

Possibly atomically sets the element at index `i` to `newValue` if the element's current value == `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSetPlain(Object...)`.

Parameters:

`i` - the index

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

Since:

9

getAndIncrement

```
public final long getAndIncrement(int i)
```

Atomically increments the value of the element at index `i`, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.

Equivalent to `getAndAdd(i, 1)`.

Parameters:

`i` - the index

Returns:

the previous value

getAndDecrement

```
public final long getAndDecrement(int i)
```

Atomically decrements the value of the element at index `i`, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.

Equivalent to `getAndAdd(i, -1)`.

Parameters:

`i` - the index

Returns:

the previous value

getAndAdd

```
public final long getAndAdd(int i,
                            long delta)
```

Atomically adds the given value to the element at index `i`, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.

Parameters:

`i` - the index

`delta` - the value to add

Returns:

the previous value

incrementAndGet

```
public final long incrementAndGet(int i)
```

Atomically increments the value of the element at index `i`, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.

Equivalent to `addAndGet(i, 1)`.

Parameters:

`i` - the index

Returns:

the updated value

decrementAndGet

```
public final long decrementAndGet(int i)
```

Atomically decrements the value of the element at index `i`, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.

Equivalent to `addAndGet(i, -1)`.

Parameters:

`i` - the index

Returns:

the updated value

addAndGet

```
public long addAndGet(int i,
                      long delta)
```

Atomically adds the given value to the element at index `i`, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.

Parameters:

`i` - the index

`delta` - the value to add

Returns:

the updated value

getAndUpdate

```
public final long getAndUpdate(int i,
                               LongUnaryOperator updateFunction)
```

Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the element at index `i` with the results of applying the given function, returning the previous value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads.

Parameters:

`i` - the index

`updateFunction` - a side-effect-free function

Returns:

the previous value

Since:

1.8

updateAndGet

```
public final long updateAndGet(int i,
                               LongUnaryOperator updateFunction)
```

Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the element at index `i` with the results of applying the given function, returning the updated value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads.

Parameters:

`i` - the index

`updateFunction` - a side-effect-free function

Returns:

the updated value

Since:

1.8

getAndAccumulate

```
public final long getAndAccumulate
(int i,
 long x,
 LongBinaryOperator accumulatorFunction)
```

Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the element at index `i` with the results of applying the given function to the current and given values, returning the previous value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads.
The function is applied with the current value of the element at index `i` as its first argument, and the given update as the second argument.

Parameters:

`i` - the index

`x` - the update value

`accumulatorFunction` - a side-effect-free function of two arguments

Returns:

the previous value

Since:

1.8

accumulateAndGet

```
public final long accumulateAndGet
(int i,
 long x,
 LongBinaryOperator accumulatorFunction)
```

Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the element at index `i` with the results of applying the given function to the current and given values, returning the updated value. The function should be side-effect-free, since it

may be re-applied when attempted updates fail due to contention among threads.
The function is applied with the current value of the element at index `i` as its first argument, and the given update as the second argument.

Parameters:

`i` - the index

`x` - the update value

`accumulatorFunction` - a side-effect-free function of two arguments

Returns:

the updated value

Since:

1.8

toString

```
public String toString()
```

Returns the String representation of the current values of array.

Overrides:

`toString` in class `Object`

Returns:

the String representation of the current values of array

getPlain

```
public final long getPlain(int i)
```

Returns the current value of the element at index `i`, with memory semantics of reading as if the variable was declared non-`volatile`.

Parameters:

`i` - the index

Returns:

the value

Since:

9

setPlain

```
public final void setPlain(int i,
                           long newValue)
```

Sets the element at index `i` to `newValue`, with memory semantics of setting as if the variable was declared non-`volatile` and non-`final`.

Parameters:

`i` - the index

`newValue` - the new value

Since:

9

getOpaque

```
public final long getOpaque(int i)
```

Returns the current value of the element at index `i`, with memory effects as specified by `VarHandle.getOpaque(Object...)`.

Parameters:

`i` - the index

Returns:

the value

Since:

9

setOpaque

```
public final void setOpaque(int i,
                            long newValue)
```

Sets the element at index `i` to `newValue`, with memory effects as specified by `VarHandle.setOpaque(Object...)`.

Parameters:

`i` - the index

`newValue` - the new value

Since:

9

getAcquire

```
public final long getAcquire(int i)
```

Returns the current value of the element at index `i`, with memory effects as specified by `VarHandle.getAcquire(Object...)`.

Parameters:

`i` - the index

Returns:

the value

Since:

9

setRelease

```
public final void setRelease(int i,
                             long newValue)
```

Sets the element at index `i` to `newValue`, with memory effects as specified by `VarHandle.setRelease(Object...)`.

Parameters:

`i` - the index

`newValue` - the new value

Since:

9

compareAndExchange

```
public final long compareAndExchange(int i,
                                     long expectedValue,
                                     long newValue)
```

Atomically sets the element at index `i` to `newValue` if the element's current value, referred to as the *witness value*, `==` `expectedValue`, with memory effects as specified by `VarHandle.compareAndExchange(Object...)`.

Parameters:

`i` - the index

`expectedValue` - the expected value

`newValue` - the new value

Returns:

the *witness value*, which will be the same as the expected value if successful

Since:

9

compareAndExchangeAcquire

```
public final long compareAndExchangeAcquire(int i,
                                             long expectedValue,
                                             long newValue)
```

Atomically sets the element at index `i` to `newValue` if the element's current value, referred to as the *witness value*, `==` `expectedValue`, with memory effects as specified by `VarHandle.compareAndExchangeAcquire(Object...)`.

Parameters:

`i` - the index

`expectedValue` - the expected value

`newValue` - the new value

Returns:

the *witness value*, which will be the same as the expected value if successful

Since:

9

compareAndExchangeRelease

```
public final long compareAndExchangeRelease(int i,
                                             long expectedValue,
                                             long newValue)
```

Atomically sets the element at index `i` to `newValue` if the element's current value, referred to as the *witness value*, `==` `expectedValue`, with memory effects as specified by `VarHandle.compareAndExchangeRelease(Object...)`.

Parameters:

`i` - the index

`expectedValue` - the expected value

`newValue` - the new value

Returns:

the *witness value*, which will be the same as the expected value if successful

Since:

9

weakCompareAndSetVolatile

```
public final boolean weakCompareAndSetVolatile(int i,
                                               long expectedValue,
                                               long newValue)
```

Possibly atomically sets the element at index `i` to `newValue` if the element's current value == `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSet(Object...)`.

Parameters:

`i` - the index

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

Since:

9

weakCompareAndSetAcquire

```
public final boolean weakCompareAndSetAcquire(int i,
                                              long expectedValue,
                                              long newValue)
```

Possibly atomically sets the element at index `i` to `newValue` if the element's current value == `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSetAcquire(Object...)`.

Parameters:

`i` - the index

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

Since:

9

weakCompareAndSetRelease

```
public final boolean weakCompareAndSetRelease(int i,
                                              long expectedValue,
                                              long newValue)
```

Possibly atomically sets the element at index `i` to `newValue` if the element's current value == `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSetRelease(Object...)`.

Parameters:

`i` - the index

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

Since:

9

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.